

Devoir surveillé terminal

Durée 2h
(Corrections)

Nous souhaitons développer un système permettant d'exécuter des fonctions à distance par des processus quelconques. Un programme, appelé l'**exécuteur**, est en mesure de recevoir des requêtes distantes qui correspondent à une demande d'exécution d'une fonction : le nom de la fonction ainsi que la valeur de ses paramètres. La fonction est alors exécutée puis le résultat est envoyé à l'émetteur de la requête.

Les **processus locaux** qui désirent réaliser une exécution distante, passent par l'intermédiaire d'un **proxy local**. Celui-ci a pour but de mettre en contact ces processus avec un ou plusieurs exécuteurs distants. Pour cela, le proxy local établit une connexion vers chaque exécuteur connu : pour chaque connexion, un fils dédié est créé. Chaque fils enregistre toutes les fonctions disponibles, récupérées auprès de l'exécuteur, dans un segment de mémoire partagée.

Les processus locaux s'attachent au segment de mémoire partagée pour obtenir les noms des fonctions exécutables et les paramètres nécessaires, ainsi que le PID du fils du proxy correspondant. Lorsqu'ils souhaitent effectuer une exécution d'une fonction distante, les processus locaux envoient leur requête dans une file de messages. Chaque fils du proxy peut alors recevoir les requêtes qui lui sont destinées. Les résultats reçus sont envoyés dans la même file de messages.

Pour simplifier, nous caractérisons une fonction par son nom (maximum 20 caractères), le nombre de ses paramètres, la taille en octets de chacun et la taille en octets du résultat. Le type des paramètres est supposé connu par le processus qui souhaite exécuter la fonction.

1°) **Enregistrement et attachement des exécuteurs** (5,5 points). Au démarrage du proxy local, celui-ci lit depuis un fichier binaire les adresses de chaque exécuteur (adresse IPv4 + numéro de port). Une application, nommée **reg**, permet à l'utilisateur d'enregistrer les adresses de nouveaux exécuteurs dans ce fichier. L'application prend en arguments le nom du fichier binaire, l'adresse IPv4 et le numéro de port du nouvel exécuteur. Avant de l'ajouter, elle vérifie que l'adresse n'est pas déjà présente dans le fichier.

a) Donnez une structure en C, nommée **adresse_t**, permettant de représenter l'adresse d'un exécuteur. Expliquez votre choix et donnez la taille en mémoire de cette structure.

Solution : (0,5 point). Une adresse IPv4 au format réseau est représentée par 4o (soit un **uint32_t** ou un **int**). Le numéro de port par un entier court (soit un **in_port_t** ou un **short**). La taille est ici de 4 octets et 2 octets, mais avec l'alignement, soit 8 octets. Voici une structure possible :

```
typedef struct {
    uint32_t adresse;    /* Adresse IPv4 */
    in_port_t port;      /* Numéro de port */
} adresse_t;
```

b) Expliquez en quelques phrases comment vérifier qu'une adresse n'est pas déjà présente dans le fichier.

Solution : (0,5 point). On ouvre le fichier en lecture et on se place au début. On lit chaque adresse tant qu'on est pas à la fin du fichier. Puis on compare chaque adresse lue avec celle recherchée. Ce qui est important, c'est d'expliquer qu'il faut lire chaque adresse car la comparaison ne peut pas être réalisée directement dans le fichier.

c) Nous supposons l'existence de la fonction **int est_presente(int fd, adresse_t adr)** qui retourne 1 si l'adresse existe dans le fichier, 0 sinon. Le paramètre **fd** correspond au descripteur du fichier binaire ouvert en lecture/écriture et le paramètre **adr** l'adresse que l'on souhaite vérifier. Écrivez la procédure **void ajouter_adresse(char *nomFichier, adresse_t adr)** qui permet d'ajouter la nouvelle adresse à la fin

du fichier si elle n'est pas déjà présente. Si le fichier n'existe pas, il est créé. Le paramètre `nomFichier` est le nom du fichier binaire et le paramètre `adr` la nouvelle adresse.

Solution : (1 point). Voici une solution possible.

```
void ajouter_adresse(char *nomFichier, adresse_t adr) {
    int fd;
    adresse_t tmp;

    fd = open(nomFichier, O_CREAT | O_RDWR, S_IRUSR | S_IWUSR);
    if(est_presente(fd, adr) == 0) {
        lseek(fd, 0L, SEEK_END);
        write(fd, &adr, sizeof(adresse_t));
    }
    close(fd);
}
```

d) Nous supposons l'existence de la procédure `void fils(adresse_t adr)` qui correspond à la routine principale de chaque processus fils du proxy local. Elle prend en paramètre l'adresse d'un exécuteur et établit une connexion TCP avec celui-ci. Elle récupère ensuite toutes les fonctions de l'exécuteur qu'elle enregistre dans le segment de mémoire partagée. Puis elle se met en attente de requêtes de la part de processus locaux depuis la file de messages. Écrivez la fonction `void creation_fils(char *nomFichier)` qui prend en paramètre le nom du fichier contenant les adresses des exécuteurs et qui crée un fils pour chacune. Chaque fils exécute la procédure `fils`.

Solution : (1,5 points). Voici une solution possible.

```
void creation_fils(char *nomFichier) {
    int fd, n;
    adresse_t tmp;

    fd = open(nomFichier, O_RDONLY);
    if(fd != -1) {
        lseek(fd, 0L, SEEK_SET); /* Inutile */
        while((n = read(fd, &tmp, sizeof(adresse_t))) != 0) {
            if(fork() == 0)
                fils(tmp);
        }
        close(fd);
    }
}
```

e) Nous supposons que le programme principal du proxy prend en argument le nom du fichier contenant les adresses des exécuteurs. Il crée les fils (avec la fonction `creation_fils`) puis attend que l'utilisateur presse CTRL+C. Donnez le code du `main`.

Solution : (2 points). Voici une solution possible.

```
int main(int argc, char *argv[]) {
    siginfo_t info;
    sigset_t ensemble, sigs_new, sigs_old;

    creation_fils(argv[1]);

    /* Blocage des signaux */
    sigfillset(&sigs_new);
    sigprocmask(SIG_BLOCK, &sigs_new, &sigs_old);

    /* Attente du signal SIGINT (pour CTRL+C) */
    sigemptyset(&ensemble);
    sigaddset(&ensemble, SIGINT);
    sigwaitinfo(&ensemble, &info);
}
```

```

/* Déblocage des signaux */
sigprocmask(SIG_BLOCK, &sig_old, NULL);

return EXIT_SUCCESS;
}

```

2°) Communications réseaux (4 points).

a) Expliquez, à votre avis, pourquoi le mode connecté est préférable au mode non connecté dans cette application.

Solution : (0,5 point). Ici, des échanges multiples sont réalisés. Il est donc plus efficace d'utiliser le mode connecté qui permettra de vérifier automatiquement qu'aucun message n'est perdu (c'est TCP qui s'en charge).

b) Combien de sockets sont nécessaires sur chaque proxy (tous ses fils confondus)? Et sur un exécuteur (tous ses fils confondus)?

Solution : (0,5 point) Une socket pour chaque fils du proxy. Donc, n sockets avec n le nombre d'exécuteurs. L'exécuteur se met en attente de connexions sur une socket de connexion. Chaque connexion produit une socket de communication. Donc, $m + 1$ sockets avec m le nombre de proxys connectés.

c) Donnez tous les échanges possibles entre les fils et les exécuteurs distants. Vous spécifierez les données échangées (type, taille, rôle).

Solution : (1,5 points) Lorsque la socket est créée entre un fils du proxy local et un fils de l'exécuteur, ils doivent échanger les prototypes des fonctions. Chaque fonction est caractérisée par son nom (max. 200 donc chaîne constante suffisante), le nombre de paramètres et la taille de chacun, ainsi que la taille des résultats. Donc :

- Envoi du nombre de fonctions (*int*)
- Pour chaque fonction : envoi du nom de la fonction ($20 \times \text{char}$), du nombre de paramètres (*int*), la taille de chacun ($\text{int} \times \text{nombre de paramètres}$), taille du résultat (*int*).

Pour l'exécution du fonction :

- Envoi du nom de la fonction ($\text{char} \times 20$)
- Envoi des valeurs des paramètres : pour chaque $\text{char} \times \text{taille du paramètre}$
- Réception du résultat ($\text{char} \times \text{taille du résultat}$)

d) Donnez la partie du code de la procédure `files` (correspondant à un fils d'un proxy) qui établit une connexion TCP avec l'exécuteur dont l'adresse est passée en paramètre de la procédure.

Solution : (1,5 points) Voici la portion de code (en considérant que la structure `adresse_t` contient déjà les adresses IPv4 et port au format réseau :

```

fd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP));

memset(&adresse, 0, sizeof(struct sockaddr_in));
adresse.sin_family = AF_INET;
adresse.sin_addr.s_addr = adr.adresse;
adresse.sin_port = adr.port;

connect(fd, (struct sockaddr*)&adresse, sizeof(adresse));

```

3°) Segment de mémoire partagée (5 points).

a) Le segment de mémoire partagée contient les "signatures" de chaque fonction, ajoutées par les différents fils du proxy local. Donnez une structure en C, nommée `fonction_t`, permettant de représenter une signature de fonction en mémoire.

Solution : (0,5 point) Voici une solution :

```
typedef struct {
    char *nom; /* Nom de la fonction */
    int nombreParametres; /* Nombre de paramètres */
    int *tailleParametres; /* Tailles des paramètres */
    int tailleResultat; /* Taille du résultat */
    pid_t pid; /* PID du fils */
} fonction_t;
```

b) Expliquez comment stocker cette structure dans le segment de mémoire partagée.

Solution : (0,5 point) Les données sont simplement stockées de manière contigüe dans le segment.

c) Proposez une solution permettant d'ajouter des fonctions dans le segment. Elle devra permettre aussi de parcourir les fonctions présentes dans le segment.

Solution : (1 point) Les signatures des fonctions faisant des tailles différentes, il est difficile de savoir où est placée la dernière fonction, à moins de lire toutes les fonctions déjà présentes. Il est possible d'ajouter la position libre actuelle, une table de position (nom indexé avec les positions dans le segment), etc. Dans tous les cas, il est nécessaire d'avoir le nombre de fonctions présentes dans le segment.

d) Donnez le code de la fonction `void* attacher_segment(key_t cle)` permettant d'attacher le segment de mémoire partagée en mémoire. Le paramètre `cle` est la clé IPC du segment. Si le segment n'existe pas, la fonction retourne `NULL`.

Solution : (1 point) Voici une solution :

```
void* attacher_segment(key_t cle) {
    int shmid = shmget((key_t)CLE, 0, S_IRUSR | S_IWUSR);
    if (shmid == -1)
        return NULL;
    else
        return shmat(shmid, NULL, 0);
}
```

e) Écrivez la fonction `void ajouter_fonction(fonction_t fct, void *position)` qui permet à un fils du proxy d'ajouter la fonction à la position spécifiée dans le segment de mémoire partagée. Le paramètre `fct` correspond à la signature de la fonction, le paramètre `position` correspond à la position à laquelle ajouter la fonction (il s'agit bien de la position où ajouter la fonction et non pas de l'adresse du segment de mémoire partagée).

Solution : (2 points) Voici une solution :

```
void ajouter_fonction(fonction_t fct, void *position) {
    char *pos = position;

    memcpy(fct.nom, pos, sizeof(char) * 20);
    pos += 20;
    memcpy(&fct.nombreParametres, pos, sizeof(int));
    pos += sizeof(int);
    memcpy(fct.tailleParametres, pos, sizeof(int) * fct.nombreParametres);
    pos += sizeof(int) * fct.nombreParametres;
    memcpy(&fct.tailleResultat, pos, sizeof(int));
    pos += sizeof(int);
    memcpy(&fct.pid, pos, sizeof(pid_t));
}
```

4°) **File de messages** (2 points).

a) Proposez une solution permettant aux processus locaux d'envoyer leurs requêtes dans la file de messages, spécifiquement à un fils du proxy et de pouvoir lire les réponses.

Solution : (0,5 points). Lorsqu'un processus local souhaite exécuter une fonction, il envoie une requête dont le type correspond au PID du fils approprié. Il ajoute son propre PID à la requête. Les fils du proxy attendent des messages dont le type est leur PID. Lorsqu'ils lisent la requête, ils récupèrent le PID du processus local. Le type de la réponse correspond au PID du processus local.

b) Proposez la structure `resultat_t` correspondant au message envoyé par un fils du proxy à un processus local. Précisez les champs utilisés et donnez les limites éventuelles dues à votre choix.

Solution : (1 point). Le résultat ne peut pas faire plus de `TAILLE_MAX_RESULTAT`. Voici un exemple :

```
typedef struct {
    long PID;
    char resultat[TAILLE_MAX_RESULTAT];
} resultat_t;
```

c) Donnez le code de la fonction `void envoyer_resultat(int msgid, resultat_t resultat)` permettant d'envoyer un résultat dans la file de messages. Le paramètre `msgid` est l'identifiant de la file de messages et `resultat` le résultat à envoyer.

Solution : (0,5 points). Voici une solution :

```
void envoyer_resultat(int msgid, resultat_t resultat) {
    msgsnd(msgid, &resultat, sizeof(resultat_t) - sizeof(long), 0);
}
```

5°) Concurrency (3,5 points).

a) Expliquez tous les problèmes de concurrence qui doivent être gérés dans le système.

Solution : (1 point). Les seuls problèmes de concurrence concernent le segment de mémoire partagée. En effet, les fils sont créés simultanément et peuvent donc ajouter simultanément des fonctions. De plus, les processus locaux peuvent consulter le segment avant qu'il ne soit complété.

b) Combien de sémaphores sont nécessaires ? Expliquez le rôle de chacune et comment l'utiliser. Aucun processus ne doit être bloqué inutilement.

Solution : (1 point). Un seul sémaphore peut être nécessaire pour permettre l'ajout par un seul fils du proxy local. Par contre, pendant ce temps, les processus locaux ne peuvent accéder au segment local. Il est possible d'utiliser un deuxième sémaphore permettant de compter le nombre de lectures en cours et un troisième permettant de compter le nombre de demandes d'écriture.

Lorsqu'un processus local souhaite lire dans le segment, il vérifie que le nombre de demandes d'écriture est à 0 (bloqué sinon) et il incrémente le nombre de lectures de 1. Une fois la lecture terminée, il décrémente le nombre de lectures de 1. Lorsqu'un fils du proxy souhaite écrire dans le segment, il ajoute le nombre de demandes d'écriture de 1, attend que le nombre de lectures soit à 1 et tente de bloquer le sémaphore de modification. À la fin, il relâche le sémaphore de modification et décrémente le nombre de demandes d'écritures.

c) Donnez le code permettant de créer le tableau de sémaphores nécessaire et de l'initialiser. La clé IPC correspond à la constante `CLE_SEM`. Si le tableau existe déjà, il doit être supprimé avant de créer le nouveau tableau.

Solution : (1,5 point). Voici le code :

```
int semid;
unsigned short val[2] = {1, 0, 0};
if((semid=semget((key_t)CLE_SEM, 3, S_IRUSR | S_IWUSR | IPC_CREAT | IPC_EXCL)) == -1) {
    if(errno == EEXIST) {
        semid = semget((key_t)CLE_SEM, 3, S_IRUSR | S_IWUSR);
        semctl(semid, 0, IPC_RMID);
        semid = semget((key_t)CLE_SEM, 3, S_IRUSR | S_IWUSR | IPC_CREAT | IPC_EXCL);
    }
}
semctl(semid, 0, SETALL, val);
```